

# ENGR 102 Summer Bridge

## Day 13 Student Workbook

Loop Design, Break, Continue, and Nested Tracing

Wednesday, July 22, 2026 • 50 points

|      |  |     |  |         |  |
|------|--|-----|--|---------|--|
| Name |  | UIN |  | Section |  |
|------|--|-----|--|---------|--|

**Boolean-operator convention.** Python operators appear as **and**, **or**, and **not**. Their lowercase spelling is required in code.

### Daily target

increasing independence

Select skip, stop, and reset behavior while tracing repeated work inside repeated work.

|                         |                                                                                  |
|-------------------------|----------------------------------------------------------------------------------|
| <b>Allowed tools</b>    | Prior tools plus break, continue, and nested for-loop tracing                    |
| <b>Support supplied</b> | Behavioral requirements and a sample input stream are supplied.                  |
| <b>You must decide</b>  | Loop form, placement of skip and stop decisions, state updates, and reset scope. |
| <b>Scaffold level</b>   | Requirements only; students select the control-flow pattern.                     |

### Scoring and completion standard

50 points

| Evidence               | Points | Secure work shows                                                   |
|------------------------|--------|---------------------------------------------------------------------|
| Integrated trace       | 10     | intermediate state, condition results, and exact output             |
| Diagnosis and repair   | 8      | actual, intended, cause, repair, and a boundary test when requested |
| Full program and tests | 32     | coherent executable logic, required output, and defended tests      |

**Independence standard.** You may use the supplied requirements and examples, but you must produce one connected solution. A collection of disconnected correct lines is not yet a complete program.

Begin with the trace, then use its evidence habits when you construct.

**Task 1. Integrated trace****10 points**

Trace nested repetition while distinguishing a skipped pass, an early stop, and a per-station reset.

```
overall = 0

for station in range(1, 3):
    station_total = 0
    for reading in [0, station + 1, 5]:
        if reading == 0:
            continue
        if reading == 5:
            break
        station_total = station_total + reading
    overall = overall + station_total
    print(station, station_total)

print(overall)
```

**Your task**

1. Give a complete reached-pass ledger that identifies the continue, break, station\_total reset, and overall update.
2. Write every output line and explain why the inner break does not end the outer loop.

**Task 2. Diagnose and repair****8 points**

The intent is to print every odd integer through 5 and then stop. The current order omits 5.

```
value = 0
while value < 8:
    value = value + 1
    if value % 2 == 0:
        continue
    if value == 5:
        break
    print(value)
```

**Your task**

1. Give actual versus intended output, identify the ordering cause, and write a minimal repair that still stops at 5.

**Task 3. Construct a complete program****32 points across Pages 4-5****Program specification**

- Repeatedly read an integer inspection code.
- Code 0 ends normal input. A negative code is ignored and processing continues.
- A code of 90 or greater is a shutdown code: stop immediately without counting or adding it.
- Each positive code below 90 is accepted. Count accepted codes and add their values.
- After the loop, print `accepted_count` and `total` on separate lines.

**Program workspace – begin the connected solution here.**

---

---

---

---

---

---

---

---

---

---

**Task 3 continued. Test and defend the program****included in 32 points**

Continue code if needed.

---

---

---

---

---

---

---

---

---

---

Required tests, expected results, and brief defense

|  |
|--|
|  |
|--|

# VIP Lab Alignment

## Bridge Day 13 • Contract 16b4e542994c

This page adds a current lab handoff while preserving the workbook's independence-ramp tasks.

### Why this page is here

**VIP Lab Day(s):** 5

**Activities:** 18.18, 18.19, 18.20, 18.21, 18.22

Select a loop form from the requirement, complete a sample/transfer pair, then finish the visibly labeled Independent Program Studio whose final build is a complete input-to-output loop program.

### Open these current notebook cells

| Activity / stable cells                                  | Notebook work                | Evidence to carry forward                         |
|----------------------------------------------------------|------------------------------|---------------------------------------------------|
| <b>18.18</b><br>18-18-work / 18-18-transfer / 18-18-cold | <b>Multiples In Range</b>    | Choose inclusive range bounds.                    |
| <b>18.19</b><br>18-19-work / 18-19-transfer / 18-19-cold | <b>Countdown To Multiple</b> | Write a while condition from a divisibility goal. |
| <b>18.20</b><br>18-20-work / 18-20-transfer / 18-20-cold | <b>Smallest Value</b>        | Initialize comparison state from real data.       |
| <b>18.21</b><br>18-21-work / 18-21-transfer / 18-21-cold | <b>Cooldown Sequence</b>     | Track a current value and a stored sequence.      |
| <b>18.22</b><br>18-22-work / 18-22-transfer / 18-22-cold | <b>Rounds To Clear Queue</b> | Model repeated queue reduction.                   |

### Readiness check before you code

1. Classify each activity as fixed traversal or condition-controlled repetition and justify one classification. \_\_\_\_\_
2. Choose one activity and list the complete state that must still be correct after the loop ends. \_\_\_\_\_

### Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook Independent Program Studio without a reveal or copied solution. Only those studio cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one .ipynb file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a .py file or create a ZIP.